SIMATS ENGINEERING
Department of Computer Science & Engineering ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO2**	**Assessment:**	**ASSESSMENT TOOL1-ANALYTICAL PROBLEM SOLVING**
Weightage:	**45%**	**Total Marks:**	**25**
CO2Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)		
Student Name:		**Reg.No.:**	
Department:		**Date:**	

ANNEXURE A

1. A processor performs arithmetic operations on signed 8-bit integers using 2's complement representation. Consider two numbers: +45 and −23. Analyze in detail how the system performs both addition and subtraction, including binary conversion, alignment of sign bits, and intermediate steps. Determine whether overflow occurs, and explain how the processor detects overflow and interprets the final result.

Signed 8-bit Arithmetic using 2's Complement (Detailed Analysis)

We are given two signed integers:

+45

−23

The processor uses **8-bit 2's complement representation** to perform arithmetic operations.

1. Binary Conversion

Step 1: Convert +45 to Binary

Decimal: +45

Binary (8-bit): 45 = 00101101

Step 2: Convert −23 to 2's Complement

Convert +23 to binary:
23 = 00010111

Take **1's complement** (invert bits): 11101000

Add 1:
11101000 + 1 = 11101001

So,
-23 = 11101001

2. Addition: (+45) + (−23)

Binary Addition

00101101 (+45)

\+ 11101001 (-23)

00010110 (Result)

Carry Handling

Carry out of MSB is discarded in 2's complement arithmetic.

Result Interpretation
00010110 = 22

Final Answer: **+22**

Overflow Check (Addition)

Overflow occurs if:

Adding two positive numbers gives a negative result OR

Adding two negative numbers gives a positive result

Here:

+45 (positive)

−23 (negative)

➡ Different signs → **NO OVERFLOW**

3. Subtraction: (+45) − (−23)

Subtraction is done as:
A - B = A + (2’s\ complement\ of\ B)

So, 45 - (-23) = 45 + 23

Step 1: Convert +23

23 = 00010111

Step 2: Perform Addition

00101101 (+45)

\+ 00010111 (+23)

01000100

Result Interpretation

01000100 = 68

✔ Final Answer: **+68**

Overflow Check (Subtraction)

Here we are adding:

+45 (positive)

+23 (positive)

Result:

+68 (positive)

Check range of 8-bit signed integers:
-128 \text{ to } +127

➡ 68 is within range → **NO OVERFLOW**

4. Sign Bit Alignment

In 8-bit signed numbers:

MSB (Most Significant Bit) = **Sign bit**

0 → Positive

1 → Negative

Number	Binary
+45	**00101101**
-23	**11101001**

Processor automatically aligns numbers since both are already 8-bit.

Method 1: Sign Bit Rule

Overflow occurs if:

Carry into MSB ≠ Carry out of MSB

Method 2: Sign Change Rule

Positive + Positive → Negative ❌ overflow

Negative + Negative → Positive ❌ overflow

Check for Our Cases

Case 1: 45 + (−23)

Different signs → **No overflow**

Case 2: 45 + 23

Same sign, result still positive → **No overflow**

6. Final Summary

Operation	Binary Result	Decimal Result	Overflow
+45 + (−23)	00010110	+22	No
+45 − (−23) = 45 + 23	01000100	+68	No

7. Conclusion

The processor uses **2's complement arithmetic** to handle both addition and subtraction.

Negative numbers are represented using **bit inversion + 1**.

Arithmetic is performed like normal binary addition.

Overflow is detected using **sign bit rules or carry comparison**.

In both operations, the results are valid and within range, so **no overflow occurs**.

This demonstrates how processors efficiently handle signed arithmetic using 2's complement representation.

2. A high-performance computing system replaces a ripple carry adder with a 4-bit carry look ahead adder (CLA) to reduce computation delay. Given two binary inputs, analyze how generate and propagate signals are formed and how carries are computed in advance. Compare the time delay of CLA with ripple carry addition and justify, with reasoning, why CLA improves speed in modern processors.

Carry Look Ahead Adder (CLA) – Detailed Analysis

A **Carry Look Ahead Adder (CLA)** is designed to overcome the delay problem of a **Ripple Carry Adder (RCA)** by computing carry signals in advance using **Generate (G)** and **Propagate (P)** concepts.

1. Basic Problem with Ripple Carry Adder (RCA)

In an RCA:

- Each bit must wait for the **previous carry**
- Carry ripples from LSB → MSB

Example Delay:

For a 4-bit adder
C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow C_4

Total delay:
T = 4 \times t_{carry}

This becomes **very slow** for large bit sizes (like 32-bit, 64-bit)

2. Concept of Generate (G) and Propagate (P)

For each bit position (i):

Generate (G_i)

G_i = A_i \cdot B_i

- Carry is **generated directly**
- Happens when both bits are 1

Propagate (P_i)

$$P_i = A_i \oplus B_i$$

- Carry is **propagated forward**
- Happens when one of the bits is 1

3. Carry Computation in CLA

Instead of waiting, CLA calculates all carries using formulas.

Carry Equations

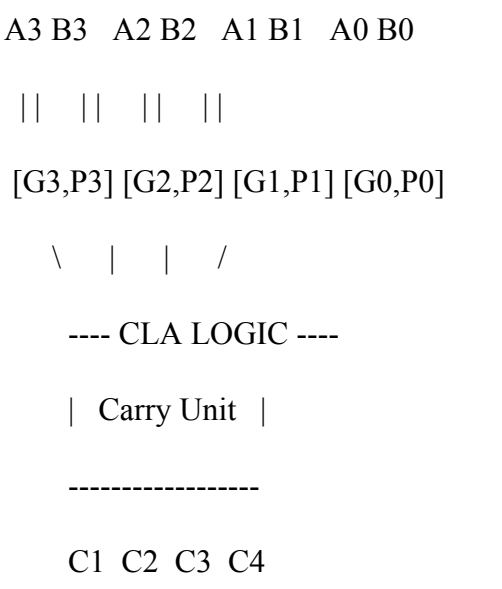
$$C_1 = G_0 + P_0 C_0$$
$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$
$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$
$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0$$

All carries are computed **in parallel (simultaneously)**

4. Sum Calculation

$$S_i = P_i \oplus C_i$$

5. CLA Block Diagram (Conceptual)



Final Sum:

S0 S1 S2 S3

6. Example (4-bit Addition)

Let:

- A = 1011
- B = 0101
- $C_0 = 0$

Step 1: Compute G and P

A	**B**	**G = AB**	**P = A ⊕ B**
1	1	1	0
1	0	0	1
0	1	0	1
1	0	0	1

Step 2: Compute Carries

C_1 = 1 + 0 = 1
C_2 = 0 + (1×1) = 1
C_3 = 0 + (1×0) + (1×1×1) = 1
C_4 = 0 + (1×0) + (1×1×0) + (1×1×1×1) = 1

Step 3: Compute Sum
S_i = P_i \oplus C_i

Final Output:
S = 0000,\quad Carry = 1

Result = **10000 (16 in decimal)**

7. Time Delay Comparison

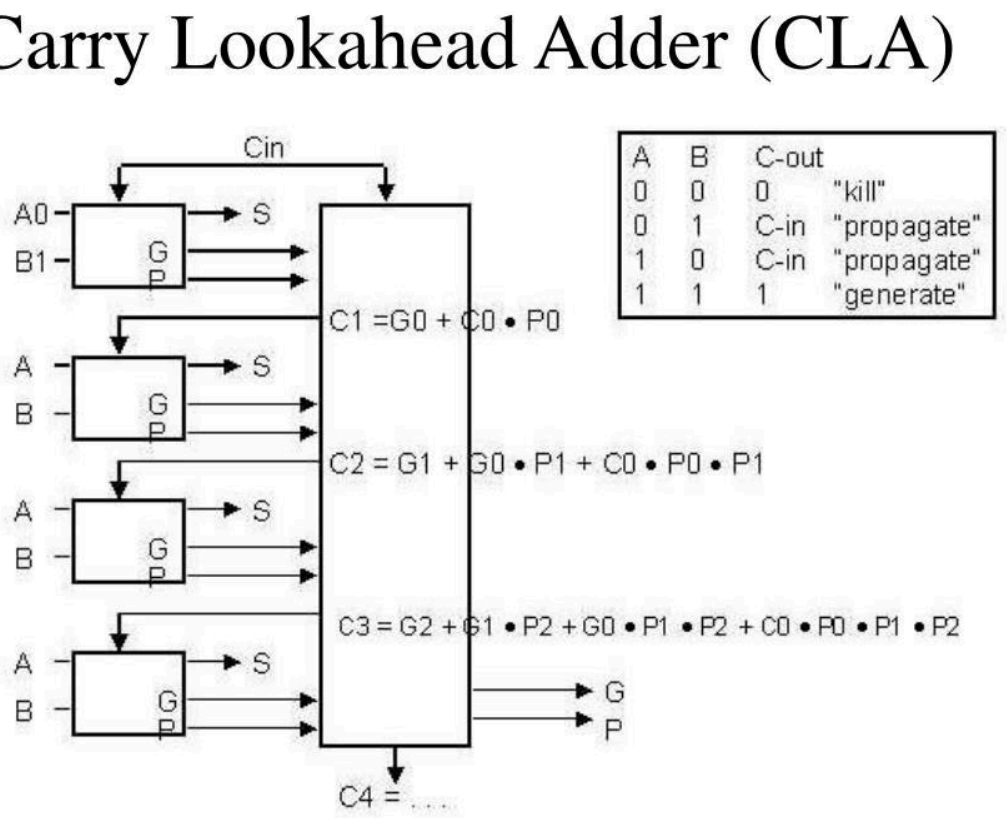
Ripple Carry Adder (RCA)

- Carry must propagate sequentially
  T_{RCA} = n \times t_{carry}

For 4-bit:
T = 4t

Carry Look Ahead Adder (CLA)

- Parallel carry computation
  T_{CLA} = t_{generate} + t_{propagate} + t_{logic}
- Approximately:
  T_{CLA} \approx 2t \text{ to } 3t



8. Why CLA is Faster

Key Reasons

- No ripple effect
  Parallel carry computation
  Reduced dependency between bits
  Faster for large bit operations

9. Comparison Table

Feature	Ripple Carry Adder	CLA
Carry Computation	Sequential	Parallel
Speed	Slow	Fast
Delay	High ($\propto$ n)	Low
Complexity	Simple	Complex
Hardware	Less	More

10. Conclusion

- CLA significantly improves performance by eliminating **carry propagation delay**
- It uses **Generate and Propagate signals** to predict carries early
- This makes CLA highly suitable for **modern processors** where speed is critical
- Though hardware complexity increases, the **speed advantage outweighs the cost**

Final Insight:
CLA is widely used in CPUs because reducing arithmetic delay directly improves **overall processor performance**.

3. In a signed multiplication operation, a processor uses Booth's algorithm to multiply two numbers, −6 and +5. Analyze the algorithm step-by-step, including the role of the accumulator, multiplier, and Booth bit, along with arithmetic shifts. Explain how the algorithm efficiently handles signed numbers and reduces the number of addition/subtraction operations compared to traditional multiplication.

Booth's Algorithm for Signed Multiplication (−6 × +5)

Booth's algorithm is an efficient method for multiplying **signed binary numbers (2's complement form)**. It reduces the number of addition/subtraction operations and handles negative numbers automatically.

1. Given Data

- Multiplicand (M) = **−6**
- Multiplier (Q) = **+5**

Using **4-bit representation**:

Convert to Binary
+5 = 0101
+6 = 0110

To get **−6 (2's complement)**:

- 1's complement → 1001
- Add 1 → **1010**
  M = -6 = 1010

2. Registers Used

Register	Purpose
A	Accumulator (initially 0000)
Q	Multiplier
M	Multiplicand
Q_{-1}	Booth bit (initially 0)

3. Initial Setup

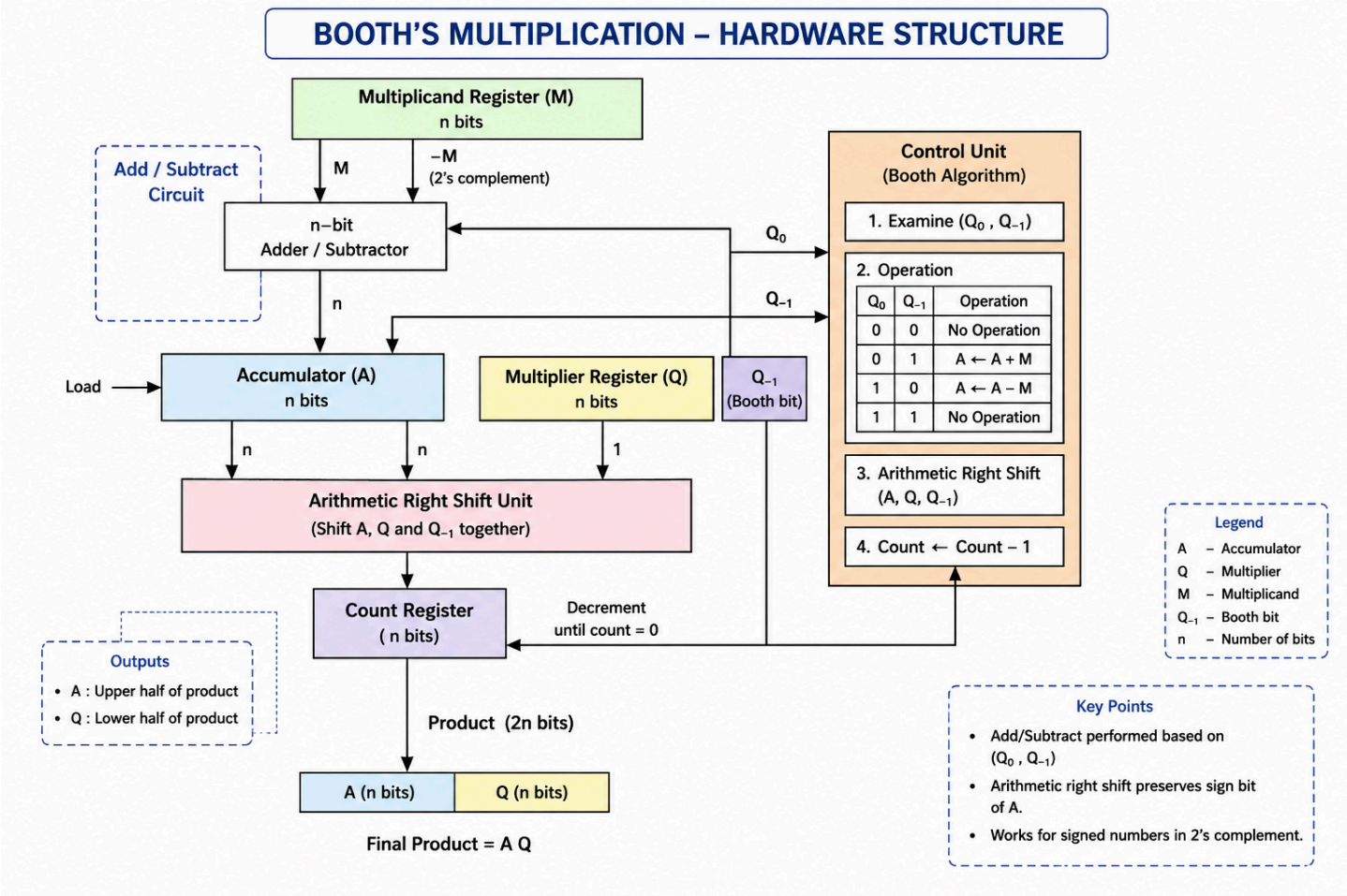
A = 0000

Q = 0101

Q-1 = 0

M = 1010 (-6)

Count = 4 (number of bits)



4. Booth's Algorithm Rules

Check **(Q_0, Q_{-1})**:

Operation

No operation

No operation

A = A − M

A = A + M

After operation → **Arithmetic Right Shift**

5. Step-by-Step Execution

Step 1Q_0 = 1, Q_{-1} = 0 → A = A − M

A = 0000 - 1010 = 0110

Shift:

A = 0011

Q = 0010

Q-1 = 1

Step 2

Q_0 = 0, Q_{-1} = 1 → **A = A + M**
A = 0011 + 1010 = 1101

Shift:

A = 1110

Q = 1001

Q-1 = 0

Step 3

$Q_0 = 1, Q_{-1} = 0 \rightarrow$ **A = A − M**
A = 1110 - 1010 = 0100

Shift:

A = 0010

Q = 0100

Q-1 = 1

Step 4

$Q_0 = 0, Q_{-1} = 1 \rightarrow$ **A = A + M**
A = 0010 + 1010 = 1100

Shift:

A = 1110

Q = 0010,Q-1 = 0

6. Final Result

Combine A and Q:
Result = A Q = 1110\ 0010

This is an 8-bit result.

Convert to Decimal

Since MSB = 1 → Negative number

Take 2's complement:

11100010 → 00011101 + 1 = 00011110= 30

So,
Result = -30

Correct answer:
-6 \times 5 = -30

7. Role of Components

Accumulator (A)

- Stores intermediate results
- Used for addition/subtraction

Multiplier (Q)

- Holds multiplier bits
- Controls operations via Q_0

Booth Bit (Q_{-1})

- Stores previous bit
- Helps detect transitions (01, 10)

Arithmetic Right Shift

- Preserves sign bit
- Shifts A, Q, Q_{-1} together

8. Why Booth's Algorithm is Efficient

Key Idea:

It reduces operations by analyzing **bit patterns**

Example

Binary:

0111110

Normal multiplication:

- Many additions

Booth's method:

- Detects pattern → converts to:
  +2^n - 2^m
- Fewer operations

9. Advantages Over Traditional Multiplication

Feature	Traditional	Booth
Signed Numbers	Complex	Easy
Operations	More	Reduced
Speed	Slower	Faster
Efficiency	Low	High

10. Conclusion

- Booth's algorithm efficiently handles **signed multiplication**
- Uses $\mathbf{Q_0}$ **and** $\mathbf{Q_{-1}}$ **comparison** to reduce unnecessary operations
- Performs **fewer additions/subtractions**
- Ideal for **modern processors** where speed and efficiency are important

Final Insight:
Booth's algorithm improves performance by **minimizing arithmetic operations**, especially when the multiplier contains consecutive 1's.

4. A processor is required to perform division of two binary numbers (e.g., 13 ÷ 3) using both restoring and non-restoring division algorithms. Analyze both methods step-by-step, showing intermediate remainders and quotient formation. Compare the two approaches in terms of number of steps, complexity, and efficiency, and explain why non-restoring division is often preferred in modern systems.

Binary Division using Restoring and Non-Restoring Algorithms (13 ÷ 3)

We will divide:
13 \div 3

1. Binary Conversion

13 = 1101 \quad (Dividend)
3 = 0011 \quad (Divisor)

Registers used:

- **A (Accumulator / Remainder)**
- **Q (Quotient / Dividend)**
- **M (Divisor)**

Initial:

A = 0000

Q = 1101

M = 0011

n = 4 bits

2. Restoring Division Algorithm

Concept

- Subtract divisor from A
- If result is negative → restore (add back M)
- Set quotient bit accordingly

Step-by-Step

Step 1

Shift left (A,Q):

A = 0001 Q = 1010

A = A − M:

0001 - 0011 = 1110 (negative)

Restore:

A = 0001

Q0 = 0

Step 2

Shift:

A = 0011 Q = 0100

A = A − M:

0011 - 0011 = 0000

Positive → Q0 = 1

Step 3

Shift:

A = 0000 Q = 1000

A = A − M:

0000 - 0011 = 1101 (negative)

Restore:

A = 0000

Q0 = 0

Step 4

Shift:

A = 0001 Q = 0000

A = A − M:

0001 - 0011 = 1110 (negative)

Restore:

A = 0001

Q0 = 0

Final Result (Restoring)

Quotient (Q) = 0100 = 4

Remainder (A) = 0001 = 1

13 ÷ 3 = 4 remainder 1

3. Non-Restoring Division Algorithm

Concept

- Avoid restoring step
- If A becomes negative → next step add M
- If A positive → subtract M

Step-by-Step

Step 1

Shift:

A = 0001 Q = 1010

A = A − M:

0001 - 0011 = 1110 (negative)

Q0 = 0

Step 2

Shift:

A = 1101 Q = 0100

A is negative → A = A + M:

1101 + 0011 = 0000

Q0 = 1

Step 3

Shift:

A = 0000 Q = 1000

A positive → subtract:

0000 - 0011 = 1101 (negative)

Q0 = 0

Step 4

Shift:

A = 1011 Q = 0000

A negative → add:

1011 + 0011 = 1110

Q0 = 0

Final Correction

A is negative → add M:

1110 + 0011 = 0001

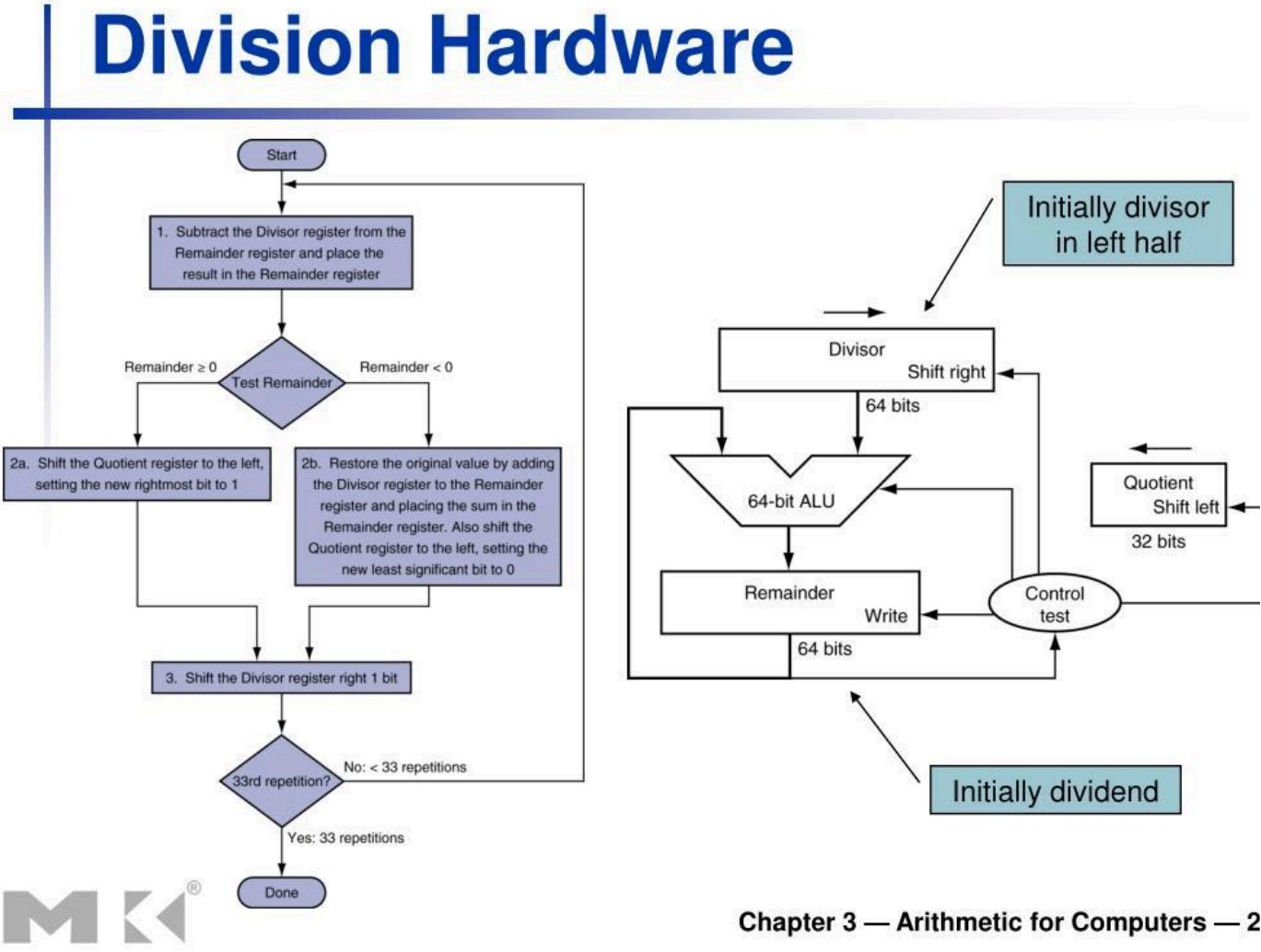
Final Result (Non-Restoring)

Quotient (Q) = 0100 = 4

Remainder (A) = 0001 = 1

Same correct result

4. Division Hardware Structure (Conceptual Diagram)



5. Comparison

Feature	Restoring Division	Non-Restoring Division
Restore Step	Required	Not required
Operations	More	Fewer
Speed	Slower	Faster
Complexity	Simple	Slightly complex
Efficiency	Lower	Higher

6. Number of Steps Analysis

- Both take **n iterations (4 here)**
- But:
    - Restoring → extra **restore operation**
    - Non-restoring → avoids it

Hence fewer arithmetic operations overall

7. Why Non-Restoring is Preferred

- Eliminates restore step → reduces delay
  Fewer additions/subtractions
  Better hardware efficiency
  Faster execution in processors

8. Conclusion

- Both algorithms give the same correct result
- **Restoring division** is easier to understand but slower
- **Non-restoring division** improves performance by removing unnecessary operations

- **Final Insight:**
  Modern processors prefer **non-restoring division** because it **reduces computation time and improves efficiency**, which is critical in high-speed systems.

5. A scientific application requires precise handling of real numbers using the IEEE 754 standard for floating-point representation. Given a decimal number (e.g., −13.25), analyze how it is represented in both single precision and double precision formats, including sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between two such numbers, highlighting issues like normalization, rounding, and precision loss.

IEEE 754 Floating-Point Representation (Example: −13.25)

1. Convert Decimal to Binary

13 = 1101
0.25 = 0.01

So,
$13.25 = 1101.01_2$

2. Normalize the Number

$1101.01_2 = 1.10101 \times 2^3$

3. Fields Identification

Sign (S) = 1 (negative)
Exponent (E) = 3
Mantissa (M) = 10101…

4. Single Precision (32-bit)

- 1 bit → Sign
- 8 bits → Exponent
- 23 bits → Mantissa

Bias = 127

Exponent = 3 + 127 = 130 = 10000010_2

Final Representation:
S | Exponent | Mantissa
1 | 10000010 | 10101000000000000000000

Final 32-bit:
11000001010101000000000000000000

5. Double Precision (64-bit)

- 1 bit → Sign
- 11 bits → Exponent
- 52 bits → Mantissa

Bias = 1023

Exponent = 3 + 1023 = 1026 = 10000000010_2

Final Representation:
S | Exponent | Mantissa
1 | 10000000010 | 10101000000000000000000000000000000000000000000000000

6. Floating-Point Addition

Example:
A = 13.25 = 1.10101×2^3
B = 6.5 = 1.101×2^2

Step 1: Align Exponents

$6.5 \rightarrow 0.1101 \times 2^3$

Step 2: Add Mantissas

1.10101
+0.11010
=10.01111

Step 3: Normalize

$10.01111 = 1.001111 \times 2^4$

Step 4: Adjust Exponent

New exponent = 4

Step 5: Rounding

Trim mantissa to fit bit limit (23 or 52 bits)

7. Key Concepts

Normalization:
Number must be in form 1.xxxxx × 2^E

Rounding:
Extra bits are rounded (usually round to nearest)

Precision Loss:
Occurs when adding very small number to large number

Example:
1.000000 + 0.000001 ≈ 1.000000

Overflow:
Exponent too large

Underflow:
Exponent too small

8. Summary

Single Precision:
32 bits → (1 sign, 8 exponent, 23 mantissa)

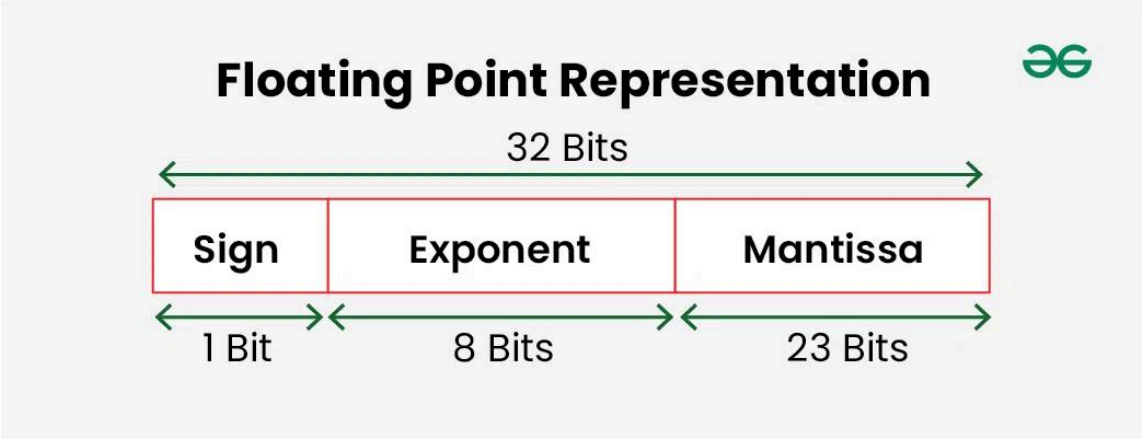
Double Precision:
64 bits → (1 sign, 11 exponent, 52 mantissa)

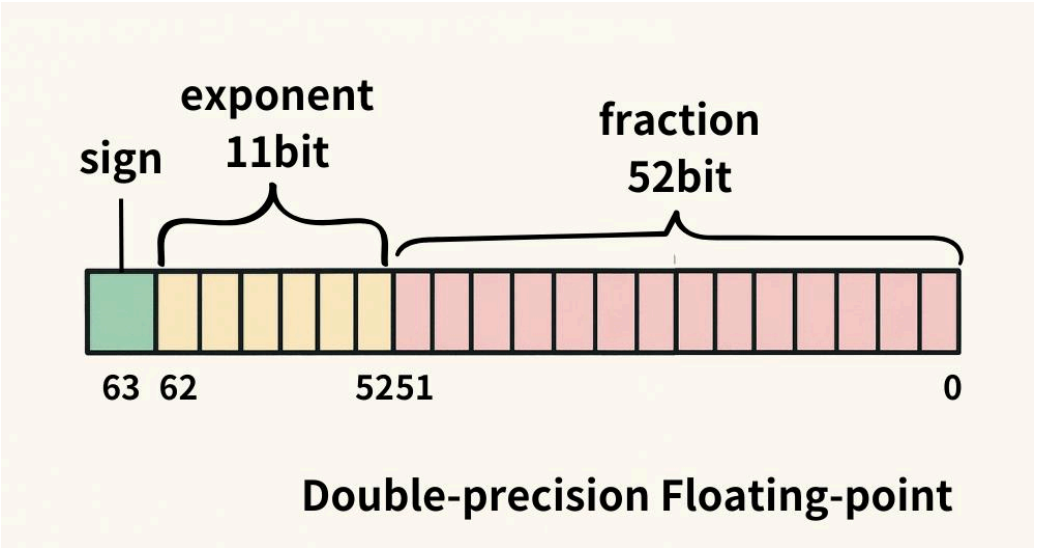
Final Answer

−13.25 in IEEE 754:

Single Precision:
11000001010101000000000000000000

Double Precision:
(64-bit representation with extended mantissa)





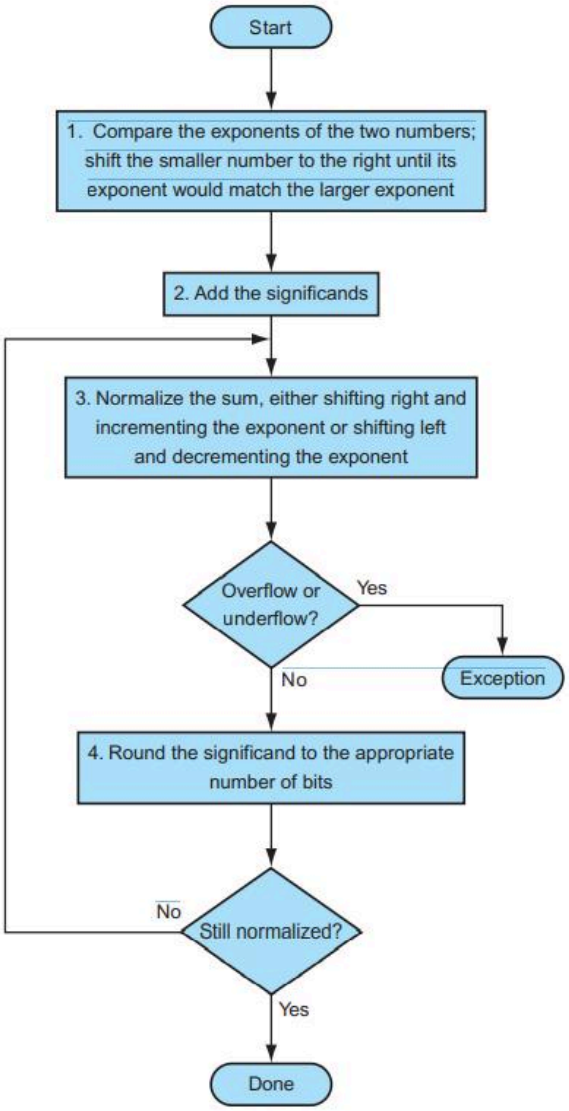


FIGURE 3.14 Floating-point addition. The normal path is to execute steps 3 and 4 once, but if rounding causes the sum to be unnormalized, we must repeat step 3.

Code of Conduct:

I ____R.mohanram/192521117____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: mohanram

MARKS ALLOCATION

	Problem Understanding(2 0%)	**Method Selection(20%)**	**Solution Process(25%)**	**Accuracy of Calculation(20%)**	**Interpretation of Results(15%)**
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation